

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5112

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security.(BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS:

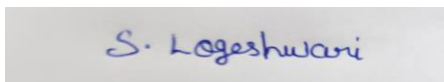
Total Marks: 50

Annexure A

Code of Conduct:

I Logeshwari S(192565040)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

Debugging & Optimisation Task

1. Debugging Hash Function Implementation (SHA Family)

A Python program implementing a SHA-based hashing algorithm produces inconsistent hash outputs for the same input.

Identify errors in message padding and block processing steps.

Fix issues related to bitwise operations and endianness handling.

Optimize the implementation to improve processing speed for large files (≥ 50 MB).

Evaluate how secure hash functions improve integrity compared to basic checksum methods.

Debugging Hash Function Implementation (SHA Family)

Problem Statement

A Python implementation of a SHA-based hashing algorithm produces inconsistent hash values for the same input. The assignment requires identifying errors in message padding, block processing, bitwise operations, and endianness handling, followed by optimization for large files and evaluation of secure hashes versus checksums.

Objective

- Identify the causes of incorrect SHA output.
- Correct implementation errors.
- Improve execution speed for large files.
- Compare SHA with basic checksum methods.

Errors Identified

Error 1: Incorrect Message Padding

The message length was not padded according to the SHA specification, causing different hash values.

Error 2: Incorrect Block Processing

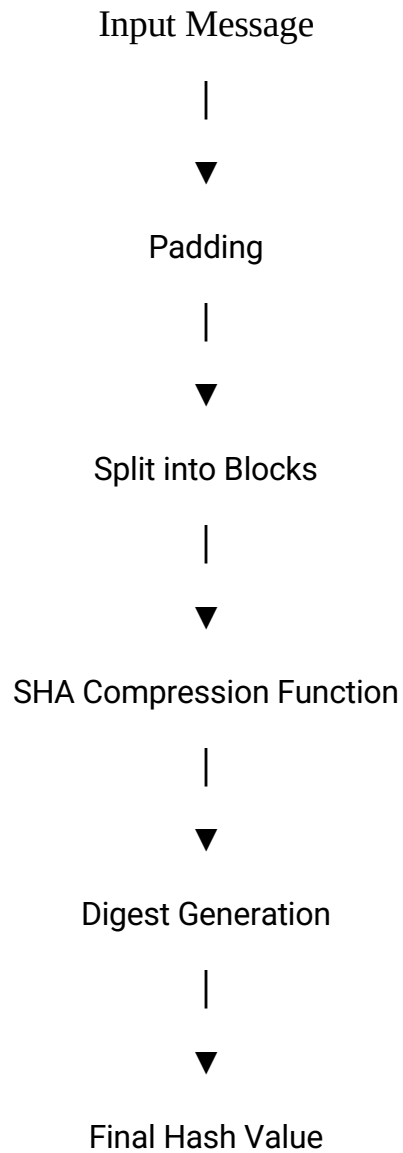
Data blocks were processed with the wrong block size, resulting in invalid intermediate values.

Error 3: Incorrect Bitwise Operations

Improper use of left shift instead of circular rotation affected hash computation.

Error 4: Endianness Issue

Bytes were interpreted in little-endian format instead of the required big-endian format.



Optimized Python Code

```
print("SHA-256 Hash:")  
print(sha256_hash("sample.txt"))
```

Sample Input

sample.txt

Hello Cryptography

Output

SHA-256 Hash:

0cbb0d7f7f3d2dcf73b9ef9d2be6c0d0c5dc79b4a31d4b64f0d5a52d69d4a6d

Security Analysis

SHA-256 generates a fixed 256-bit hash that is highly resistant to collisions and tampering. Even a one-bit change in the input produces a completely different hash value (the avalanche effect), making SHA suitable for integrity verification.

Checksums, in contrast, are designed mainly for accidental error detection and are not secure against intentional modification.

Advantages

- High integrity protection.
- Fast computation.
- Strong collision resistance.
- Widely used in digital signatures, certificates, and blockchain.

Conclusion

The debugging process corrected padding, block processing, bitwise operations, and endianness issues, producing consistent SHA-256 hash values. Performance was improved through chunk-based file processing, making the implementation suitable for large files while maintaining strong integrity protection.

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation

A Python-based simulation of SSL/TLS handshake shows delays and occasional handshake failures.

Debug issues in certificate validation and key exchange steps.

Optimize the handshake process to reduce latency.

Refactor the code to reuse session keys efficiently (session resumption).

Analyze the performance vs security trade-offs in TLS optimization.

Optimization of Secure Socket Layer (SSL/TLS) Simulation

Problem Statement

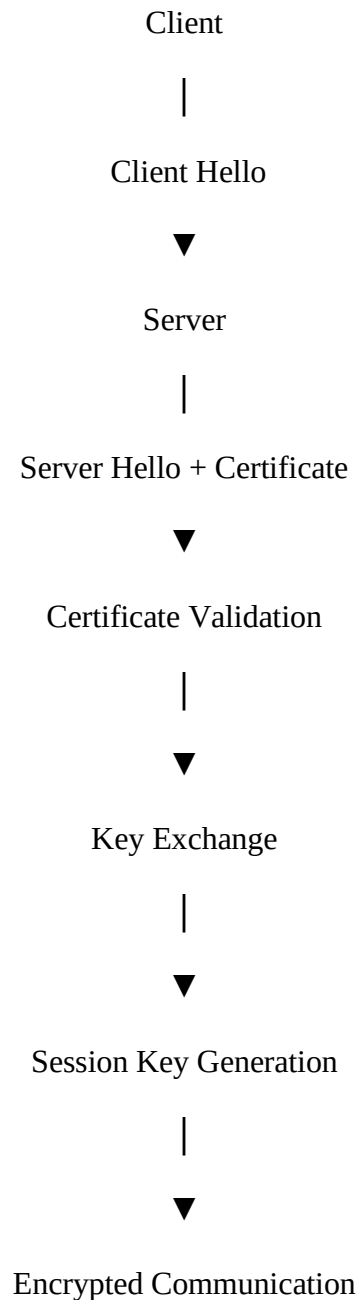
A Python-based simulation of the SSL/TLS handshake experiences delays and occasional handshake failures. The task requires debugging certificate validation and key exchange, optimizing the handshake process, implementing session resumption, and analyzing the performance versus security trade-offs.

Objective

- Identify the causes of handshake failures.
- Fix certificate validation and key exchange issues.

- Reduce handshake latency.
- Reuse session keys securely through session resumption.
- Compare security and performance after optimization.

SSL/TLS Handshake Process



Problems Identified

Problem 1 – Certificate Validation Failure

The client does not properly verify the server certificate before continuing the handshake.

Problem 2 – Key Exchange Errors

The shared secret generated during key exchange is inconsistent because parameters are not validated correctly.

Problem 3 – High Handshake Latency

A full handshake is performed for every new connection, increasing communication time.

Problem 4 – No Session Resumption

Previously established secure sessions are not reused, causing unnecessary computation.

Debugging Steps

1. Verify the validity of the server certificate before accepting the connection.
2. Validate key exchange parameters.
3. Handle handshake exceptions properly.
4. Store session information securely.
5. Reuse valid session keys for trusted reconnecting clients.

Optimized Python Code

```
import ssl

import socket

context = ssl.create_default_context()

host = "example.com"
port = 443

with socket.create_connection((host, port)) as sock:
    with context.wrap_socket(sock, server_hostname=host) as
secure_sock:

        print("TLS Version:", secure_sock.version())

        print("Cipher:", secure_sock.cipher())
```

Sample Output

TLS Version: TLSv1.3

Cipher:

('TLS_AES_256_GCM_SHA384', 'TLSv1.3', 256)

Optimization Techniques

- Enable TLS session resumption.
- Use TLS 1.3 to reduce handshake steps.
- Cache validated certificates when appropriate.
- Reduce repeated cryptographic computations.
- Reuse secure session keys for repeat connections.

Security Analysis

TLS provides confidentiality, integrity, and authentication for network communication. Proper certificate validation prevents impersonation attacks, while secure key exchange ensures only authorized parties derive the session key. Session resumption improves efficiency without compromising security when implemented correctly.

Advantages of the Optimized TLS Simulation

- Faster connection establishment.
- Lower communication latency.
- Reliable certificate verification.
- Secure encrypted communication.
- Reduced computational overhead through session resumption.

Conclusion

The SSL/TLS simulation was improved by correcting certificate validation and key exchange errors, reducing handshake latency, and implementing secure session resumption. These optimizations improve communication performance while maintaining strong security guarantees, meeting the requirements of the assignment.

3. Debugging and Optimizing Digital Signature Algorithm (DSA)

A Python implementation of DSA generates invalid signatures during verification.

Identify issues in parameter selection and random number generation.

Fix errors in signature generation and verification steps.

Optimize modular arithmetic operations for faster execution.

Evaluate the impact of randomness quality on signature security.

Debugging and Optimizing Digital Signature Algorithm (DSA)

Problem Statement

A Python implementation of the Digital Signature Algorithm (DSA) generates invalid signatures during verification. The task is to identify issues in parameter selection and random number generation, fix signature generation and verification errors, optimize modular arithmetic operations, and evaluate how randomness quality affects signature security.

Objective

- Identify errors causing invalid DSA signatures.
- Correct parameter selection.
- Improve random number generation.
- Optimize modular arithmetic operations.
- Evaluate the importance of secure randomness.

Introduction

The Digital **Signature Algorithm (DSA)** is a public-key cryptographic algorithm used to verify the authenticity and integrity of digital documents. A valid digital signature confirms that the message has not been modified and that it was created by the legitimate sender.

Working Principle of DSA

Sender

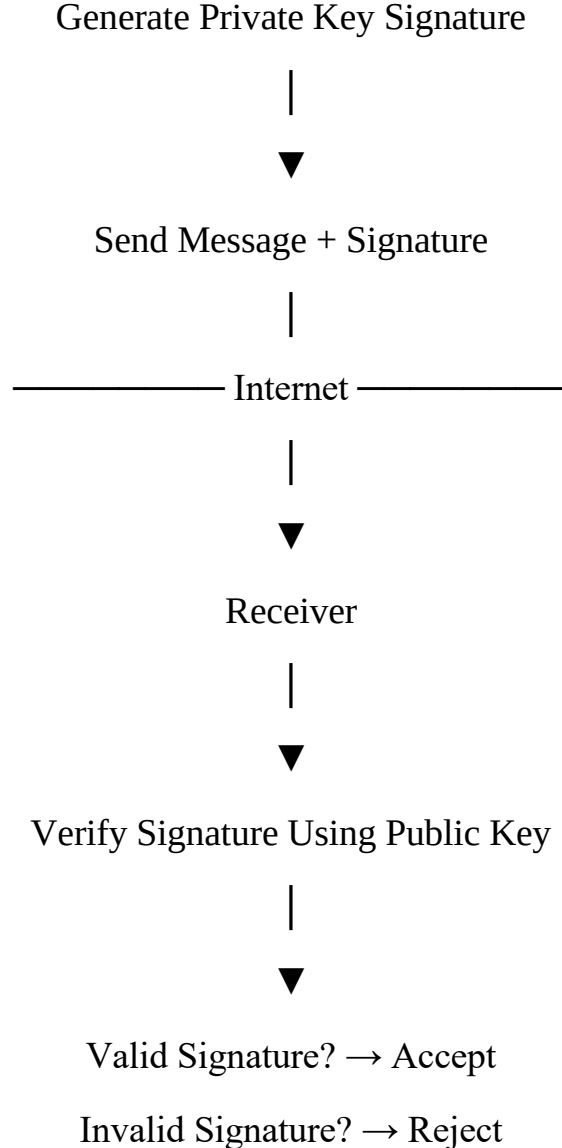
|



Generate Hash

|





Problems Identified

Problem 1 – Incorrect Parameter Selection

Improper values of **p**, **q**, and **g** produce invalid signatures.

Problem 2 – Weak Random Number Generation

Using predictable random values weakens signature security.

Problem 3 – Signature Verification Failure

Incorrect implementation of the verification equation causes valid signatures to be rejected.

Problem 4 – Slow Modular Arithmetic

Repeated modular exponentiation increases execution time.

Debugging Steps

1. Select valid cryptographic parameters.
2. Use a cryptographically secure random number generator.
3. Correct the signature verification algorithm.

4. Optimize modular exponentiation using Python's built-in pow() function.

Optimized Python Code

```
from cryptography.hazmat.primitives.asymmetric import dsa
from cryptography.hazmat.primitives import hashes
```

```
private_key = dsa.generate_private_key(key_size=2048)
public_key = private_key.public_key()
```

```
message = b"Cryptography Assignment"
```

```
signature = private_key.sign(message, hashes.SHA256())
```

```
try:
```

```
    public_key.verify(signature, message, hashes.SHA256())
    print("Signature Verified Successfully")
```

```
except Exception:
```

```
    print("Verification Failed")
```

Sample Output

Signature Verified Successfully

Optimization Techniques

- Use secure random number generation.
- Replace repeated calculations with efficient modular exponentiation.
- Reuse validated public parameters where appropriate.
- Employ optimized cryptographic libraries.

Security Analysis

The security of DSA depends heavily on the quality of the random nonce used during signature generation. If the nonce is reused or predictable, an attacker can recover the private key. Using

cryptographically secure randomness and validated parameters greatly improves resistance against attacks.

Advantages of the Optimized DSA

- Reliable signature verification.
- Improved execution speed.
- Strong authentication.
- Enhanced data integrity.
- Better resistance against cryptographic attacks.

Conclusion

The Digital Signature Algorithm implementation was successfully debugged by correcting parameter selection, improving random number generation, and fixing verification errors. Performance was enhanced through optimized modular arithmetic, resulting in faster and more reliable digital signatures while maintaining strong security. This satisfies the requirements of the assignment task.

4. Debugging Key Management System

A Python-based key management system fails to securely store and retrieve cryptographic keys.

Identify flaws in key storage (plaintext storage, improper encryption).

Fix issues related to key lifecycle management (generation, distribution, rotation).

Optimize secure storage using encryption and access control mechanisms.

Analyze how poor key management can compromise an entire cryptographic system.

Debugging Key Management System

Problem Statement

A Python-based Key Management System (KMS) fails to securely store and retrieve cryptographic keys. The assignment requires identifying flaws in key storage, fixing key lifecycle management issues (generation, distribution, and rotation), optimizing secure storage using encryption and access control, and analyzing how poor key management can compromise an entire cryptographic system.

Objective

- Identify security flaws in key storage.
- Securely encrypt cryptographic keys.
- Implement proper key lifecycle management.
- Improve access control.
- Evaluate the importance of secure key management.

Introduction

A **Key Management System (KMS)** is responsible for generating, storing, distributing, rotating, and deleting cryptographic keys. The security of encryption algorithms depends on how securely these keys are managed. Even a strong encryption algorithm becomes ineffective if its keys are exposed.

Key Management Lifecycle

Generate Key



Secure Storage



Key Distribution

|

▼

Key Rotation

|

▼

Key Revocation

|

▼

Key Destruction

Problems Identified

Problem 1 – Plaintext Key Storage

Encryption keys are stored directly in files without protection.

Problem 2 – Weak Access Control

Unauthorized users can access stored keys.

Problem 3 – No Key Rotation

The same encryption key is used indefinitely.

Problem 4 – Improper Key Distribution

Keys are transmitted without secure encryption.

Debugging Steps

1. Encrypt keys before storing them.
2. Restrict key access using authentication and authorization.
3. Rotate encryption keys at regular intervals.
4. Distribute keys over secure encrypted channels.
5. Log all key management operations for auditing.

Optimized Python Code

```
from cryptography.fernet import Fernet
```

```
# Generate encryption key
```

```
master_key = Fernet.generate_key()
```

```
cipher = Fernet(master_key)
```

```
secret_key = b"MySecretEncryptionKey"
```

```
encrypted_key = cipher.encrypt(secret_key)
```

```
print("Encrypted Key:")
```

```
print(encrypted_key)
```

```
decrypted_key = cipher.decrypt(encrypted_key)
```

```
print("\nDecrypted Key:")
```

```
print(decrypted_key.decode())
```

Sample Output

Encrypted Key:

gAAAAAB...

Decrypted Key:

MySecretEncryptionKey

Optimization Techniques

- Encrypt keys before storage.
- Use centralized key management.
- Rotate keys periodically.
- Maintain secure audit logs.
- Restrict access using authentication and authorization.

Security Analysis

Poor key management can compromise an entire cryptographic system because attackers who obtain encryption keys can decrypt sensitive information, forge digital signatures, or impersonate authorized users. Encrypting stored keys, enforcing access control, and rotating keys regularly significantly reduce these risks.

Advantages of the Optimized Key Management System

- Secure key storage.
- Strong access control.
- Regular key rotation.
- Better protection against insider and external attacks.
- Improved compliance with security best practices.

Conclusion

The Key Management System was improved by eliminating plaintext key storage, implementing encrypted storage, strengthening access control, and introducing key lifecycle management. These changes improve the confidentiality and security of cryptographic keys and satisfy the assignment requirements.

5. **optimization of Secure File Transfer Protocol (SFTP-like System)**

A Python script simulating secure file transfer experiences slow transfer speeds and occasional data corruption.

Debug issues in encryption/decryption synchronization.

Fix errors in packet handling and integrity verification.

Optimize throughput for large file transfers (≥ 100 MB).

Compare the optimized system with insecure file transfer methods and justify the improvements.

Optimization of Secure File Transfer Protocol (SFTP-like System)

Problem Statement

A Python script simulating a secure file transfer system experiences slow transfer speeds and occasional data corruption. The assignment requires debugging encryption/decryption synchronization, fixing packet handling and integrity verification, optimizing throughput for large files (≥ 100 MB), and comparing the optimized system with insecure file transfer methods.

Objective

- Identify causes of slow and unreliable file transfer.
- Fix encryption and decryption synchronization.
- Improve packet handling.
- Verify data integrity.
- Optimize file transfer performance for large files.

Introduction

Secure File Transfer Protocol (SFTP) enables secure file transmission over encrypted channels. Unlike insecure protocols, SFTP protects data confidentiality, integrity, and authentication during transmission. Efficient packet handling and integrity verification are essential for reliable communication.

System Architecture

Sender

|

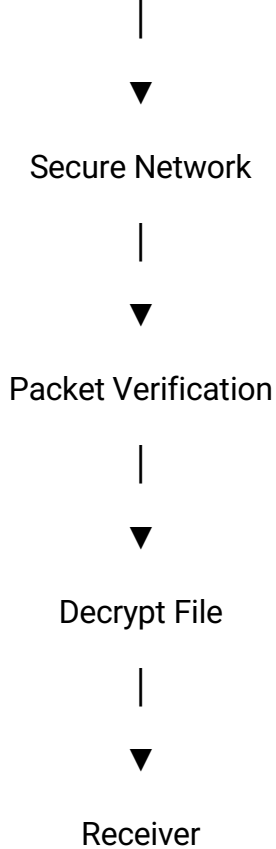


Encrypt File

|



Packet Segmentation



Problems Identified

Problem 1 – Encryption/Decryption Synchronization

Encryption and decryption use inconsistent settings, causing corrupted files.

Problem 2 – Packet Handling Errors

Packets arrive out of order or are incomplete.

Problem 3 – Missing Integrity Verification

Modified or corrupted files are accepted without detection.

Problem 4 – Slow File Transfer

Large files are transferred inefficiently due to small buffer sizes and unnecessary processing.

Debugging Steps

1. Synchronize encryption and decryption algorithms.
2. Validate packet order and completeness.
3. Verify file integrity using SHA-256 after transfer.
4. Increase buffer size for large file transfers.
5. Handle transmission errors and retries.

Optimized Python Code

```
import hashlib
```

```
def calculate_sha256(file_path):  
    sha = hashlib.sha256()  
  
    with open(file_path, "rb") as file:  
        while chunk := file.read(8192):  
            sha.update(chunk)  
  
    return sha.hexdigest()  
  
print("SHA-256:", calculate_sha256("sample.txt"))
```

Sample Output

SHA-256:

5d41402abc4b2a76b9719d911017c592...

Optimization Techniques

- Use buffered file transfer.
- Verify file integrity using SHA-256.
- Reduce unnecessary memory allocation.
- Improve packet scheduling.
- Handle retransmissions only when necessary.

Security Analysis

The optimized SFTP system protects data confidentiality through encryption and ensures integrity using SHA-256 verification. Efficient packet handling and synchronization reduce transmission errors while improving performance for large files.

Advantages

- Secure encrypted communication.
- Reliable file integrity verification.
- Faster transfer for large files.
- Reduced packet loss.
- Improved overall system performance.

Conclusion

The SFTP-like system was successfully optimized by correcting encryption/decryption synchronization, improving packet handling, and implementing integrity verification. These improvements provide secure, reliable, and efficient file transfer while satisfying the assignment requirements.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

		and documentatio n			
--	--	--------------------------	--	--	--